

Hybrid LOD Rendering between Mesh and Gaussian Splat Representations

Jerneja Krajcar

University of Ljubljana

Abstract

This seminar presents a prototype for distance-dependent transitions between mesh rendering and 3D Gaussian Splatting. The goal is to use a mesh representation for distant, low-detail views and a Gaussian representation for close-up views, while avoiding abrupt visual changes. The current main path uses Mesh2Splat exports as the Gaussian input, builds levels of detail from those exported PLY files, computes smooth transition weights from camera distance, and exposes the transition in a web viewer.

Keywords: 3D Gaussian Splatting, mesh rendering, level of detail, hybrid rendering, smooth transitions

1. Introduction

Mesheres are compact, widely supported, and reliable for low-detail views. 3D Gaussian Splatting (3DGS), as introduced by Kerbl et al. [KKLD23], is useful for high-quality appearance because it represents a scene with many semi-transparent Gaussian primitives instead of only triangle surfaces. The seminar task is therefore a hybrid rendering problem: use the scalable behaviour of meshes at distance, but switch to richer Gaussian detail when the camera moves closer.

The main difficulty is not only selecting the right representation, but changing between representations without popping, brightness jumps, or visible discontinuities. The current implementation is a prototype, not a production renderer.

2. Problem

The required system should support a seamless transition between two scene representations:

- a simplified mesh, suitable for low-detail or far-away views;
- one or more Gaussian splat representations, suitable for higher-detail close-up views.

A naive switch at a fixed distance produces visible popping because the two renderers do not necessarily match in color, coverage, silhouette, or sampling density. A better solution must make the representation change gradual and should keep neighbouring Gaussian levels spatially related, so that a denser level of detail does not look like a completely unrelated point set.

3. Related Work

3DGS provides the basic representation used for the Gaussian side of this project [KKLD23]. SuGaR aligns Gaussians with surfaces and uses this relationship for mesh extraction and high-quality mesh rendering [GL24]. This is directly relevant because it shows that mesh and Gaussian representations can be connected through surface alignment rather than treated as unrelated assets.

SplattingAvatar embeds Gaussian splats on a mesh for real-time human avatar rendering [SWL*24]. Its main target is animatable humans, but the idea of attaching or aligning Gaussian appearance to mesh structure is related to this seminar work.

Mesh2Splat [Ele25] is used as the main practical conversion path because it offers a direct way to create Gaussian splat PLY files from mesh assets. The repository also includes an experimental NVIDIA/CUDA `gsplat` training path, but this is kept separate from the Mesh2Splat workflow.

4. Implemented Solution

The current prototype is centred on an interactive web viewer and a Mesh2Splat workflow. A source mesh is loaded in the viewer and one selected Gaussian PLY file is used as the high-detail Gaussian representation. The same PLY can come from Mesh2Splat or from the experimental trained-Gaussian path.

In the current workflow, the viewer does not require a manually exported PLY file for every level of detail. Instead, it builds a deterministic nested LOD sequence from the selected Gaussian cloud. Smaller LODs are prefixes of the same ordering, and denser LODs contain all splats from coarser LODs. This makes neighbouring LODs spatially related and avoids a common popping artifact where two LODs look like unrelated point sets.

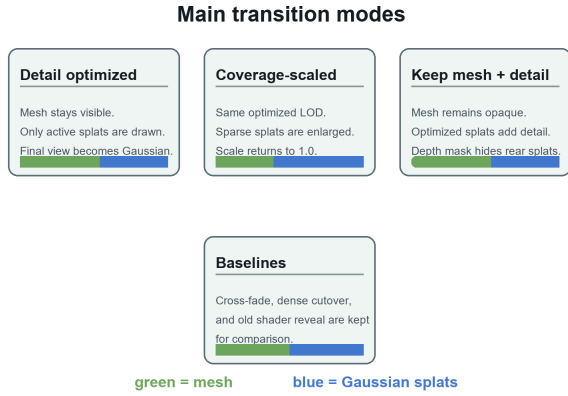


Figure 1: Transition modes exposed by the viewer. Green indicates the mesh contribution and blue indicates the Gaussian contribution.

The viewer contains several transition modes, shown schematically in Figure 1. The most useful current modes are **Detail over mesh (optimized)**, **Coverage-scaled detail over mesh**, and **Keep mesh + add detail**. The older cross-fade, dense cutover, and non-optimized detail modes are kept as baselines and debugging references.

4.1. Transition Model

All transition modes are controlled by camera distance. The viewer maps the current camera radius to a normalized transition parameter

$$t = \text{clamp} \left(\frac{r_{\text{far}} - r}{r_{\text{far}} - r_{\text{near}}}, 0, 1 \right),$$

where r is the distance from the camera to the orbit target. In the default configuration, $r_{\text{far}} = 4.0$ and $r_{\text{near}} = 1.25$ in normalized object units. When the viewer slider is used, it also moves the camera along this far-to-near range, so the slider and camera-distance transition use the same parameter.

For detail-style transitions, t is converted into a target number of active splats. This acts like a continuous LOD factor:

$$N_{\text{active}} = f(t) N_{\text{total}},$$

but $f(t)$ is not always a simple linear function. The viewer uses smoothstep ramps and artist-controlled intervals. For example, the optimized detail mode starts revealing splats late in the transition, while the coverage-scaled mode starts earlier and increases Gaussian scale temporarily to hide sparse coverage.

4.2. Gaussian LOD Sampling

The Gaussian LOD sequence is not sampled randomly. The selected Gaussian cloud is sorted once by a deterministic importance

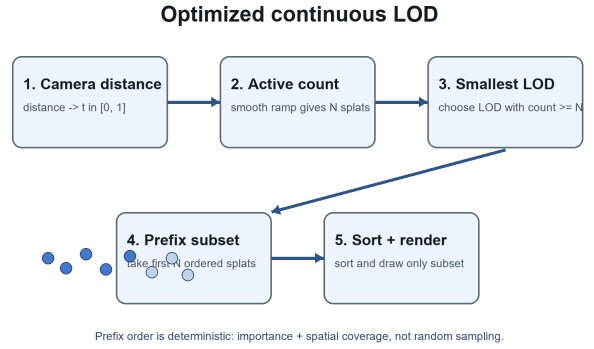


Figure 2: Optimized continuous LOD path. Camera distance is mapped to an active splat count. The viewer chooses a suitable nested LOD, takes a prefix subset, sorts only that subset, and renders only the active splats.

and spatial-coverage heuristic. Importance is based on opacity and scale, while spatial distribution is encouraged by assigning candidates to a 3D grid and taking splats from high-scoring occupied regions in a round-robin style. As a result, a small prefix should still cover different parts of the object instead of selecting only the strongest splats from one small area.

The optimized transition modes use this nested order directly. If the transition asks for N active splats, the viewer chooses the smallest prepared LOD that contains at least N splats, then slices the first N entries from that LOD. The count is rounded up to a small bucket size to avoid rebuilding WebGL buffers for every tiny camera movement. This is why the log may show a target such as 307,760 splats but a drawn subset such as 309,248 splats.

This is an important difference from the early implementation. The earlier detail mode uploaded a dense cloud and used a shader reveal mask to hide inactive splats. That was visually continuous, but it still sorted and drew the full cloud. The optimized path uploads and draws only the active subset.

4.3. Mesh and Gaussian Alignment

The mesh and Gaussian data must be in the same coordinate system before a transition can look plausible. The source mesh is normalized to a unit sphere for viewer display. For Mesh2Splat PLY files exported from the same source asset, the Gaussian positions and scales are normalized with the same mesh centre and radius. This keeps the two representations in the same object space.

For trained Gaussian files, the code also includes an alignment heuristic. It estimates robust bounds for the mesh and Gaussian cloud, tries signed axis permutations, and chooses the transform with the lowest average nearest-neighbour distance to the mesh sample. Gaussian covariance axes are transformed together with position, so anisotropic splats remain consistently oriented after alignment.

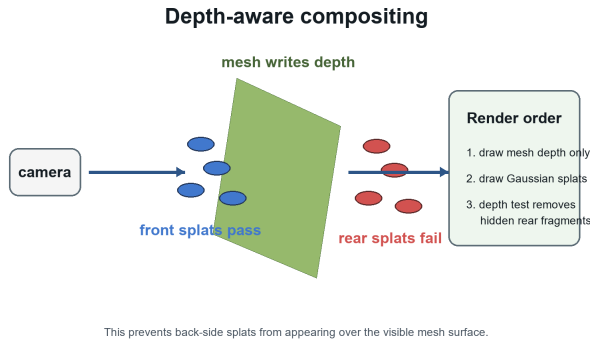


Figure 3: Depth-aware compositing. The mesh writes only depth into the Gaussian renderer; then rear splat fragments are rejected by the depth test.

4.4. Depth-Aware Compositing

The viewer renders the mesh with Three.js and Gaussian splats with a custom WebGL2 splat renderer. Without additional handling, rear-side splats can be alpha-blended over a visible front mesh surface because the mesh and Gaussian layers do not share a normal depth buffer. This caused visible discontinuities during hybrid transitions.

The current detail-style modes therefore use a depth-aware mask. Before drawing Gaussian splats, the Gaussian renderer draws the mesh geometry invisibly into its own depth buffer. Splats are then rendered with depth testing enabled. Fragments that are behind the front mesh surface fail the depth test, while front/surface splats remain visible. A small polygon offset is used when writing mesh depth so that surface splats are not removed by numerical precision issues.

4.5. Transition Modes

Cross-fade mesh to splats changes opacity between mesh and Gaussian rendering. It is simple, but the transparency change is visually weak and can show double surfaces. It remains in the viewer as a baseline.

Detail over mesh keeps the mesh visible, then reveals splats from the dense Gaussian cloud. This reduced popping compared to a hard switch, but the first version still uploaded and drew the full dense cloud.

Detail over mesh (optimized) is the best current replacement for the original detail mode. The mesh is held almost until the end of the transition, while an increasing prefix of the nested Gaussian order is rendered over it. Only the active subset is uploaded, sorted, and drawn. The mesh fades out only near the end, after the Gaussian representation is sufficiently dense.

Coverage-scaled detail over mesh uses the same optimized subset rendering as the optimized detail mode, but it starts adding splats earlier. When the active subset is sparse, the Gaussian scale is temporarily increased to cover holes. The scale boost is gradually reduced to 1.0 as the active splat count approaches the full cloud.

Keep mesh + add detail keeps the mesh fully visible and adds an optimized Gaussian detail subset on top. This mode is useful when the mesh texture is sharper than the Gaussian representation, because it preserves the original mesh appearance while adding Gaussian surface detail. It is also depth-aware, so rear splats do not draw over the mesh.

Dense LOD cutover fades directly to the densest available Gaussian LOD. It is useful as a simple reference, but it is less controlled than the optimized detail modes.

5. Alternatives Considered

Hard LOD switching would be simpler, but it directly causes popping and therefore does not satisfy the solution of the problem.

Opacity-only cross-fading was implemented first because it is easy to inspect, but it makes the transition look like a transparency effect rather than a real LOD change.

Shader-only reveal masking was useful for early experiments because it allowed continuous reveal from a dense cloud. It was later superseded by optimized subset rendering, which avoids drawing inactive splats.

Depth-aware screen-space blending would go further than the current depth mask by blending final images using per-pixel color and depth from both renderers. The implemented depth mask solves the most visible rear-splat-over-mesh artifact, but it is not yet a full image-space composition algorithm.

Direct CUDA/gsplat training was added as an experimental route for generating splats from synthetic training views on an NVIDIA/CUDA machine. It was not kept as the main path because it depends on a heavier CUDA, PyTorch, compiler, and training setup.

Full surface-aligned Gaussian optimization in the style of SuGaR would likely improve geometric consistency, but it is heavier than the current seminar prototype and would require a stronger training and reconstruction pipeline.

6. Results

The prototype currently focuses on the interactive viewer. It loads a source mesh together with a selected Gaussian PLY file and displays continuous transition modes controlled by camera distance.

The three most useful current modes have different trade-offs. **Detail over mesh (optimized)** gives the cleanest conceptual mesh-to-Gaussian transition because the mesh eventually disappears and the Gaussian representation becomes the final view. **Coverage-scaled detail over mesh** is better when sparse intermediate splats leave holes, because temporary scale expansion fills coverage gaps. **Keep mesh + add detail** is visually strong when the original mesh texture is sharper than the Gaussian PLY, because it preserves the mesh while adding Gaussian detail.

The CPU/offline renderer remains useful for validation, but it is no longer presented as the main route for high-quality Gaussian generation. The added NVIDIA/CUDA training path can produce trained Gaussian PLY files for the viewer, while GPU rasterization is still outside the current implementation.

7. Limitations and Future Work

The largest current limitation is Gaussian close-up quality. In the tested textured plant model, the original textured mesh can look sharper than the Gaussian representation. This is expected in some cases because the mesh displays the original texture directly, while the Gaussian cloud approximates appearance with a finite set of colored splats. The current WebGL renderer is also simpler than a production 3DGS rasterizer, so blur or stripe-like alpha-blending artifacts can appear.

The practical workflow still depends on externally exported Mesh2Splat PLY files. The CUDA/gsplat route adds dataset export and trainer execution support, but it is experimental and depends on an external NVIDIA training environment.

Future work should focus on four improvements: better Gaussian export or training quality, a more production-like Gaussian rasterizer, quantitative comparison on real assets, and a more stable CUDA/gsplat workflow. A later version could also bind Gaussian parameters more directly to mesh surfaces, following the direction suggested by SuGaR and SplattingAvatar.

8. Conclusion

The implemented prototype demonstrates a practical approach to smooth mesh-to-Gaussian LOD transitions. It includes mesh rendering, Mesh2Splat Gaussian input, deterministic nested Gaussian LODs, camera-distance continuous reveal, optimized subset rendering, depth-aware mesh/Gaussian compositing, and multiple transition modes for comparison.

The result is not a perfect seamless renderer, but it establishes the important components needed for the seminar task: gradual distance-based representation changes, spatially related LODs, reduced popping, and an interactive way to compare different transition strategies.

References

- [Ele25] ELECTRONIC ARTS: Mesh2Splat: Fast mesh to 3d gaussian splat conversion. <https://github.com/electronicarts/mesh2splat>, 2025. 1
- [GL24] GUÉDON A., LEPETIT V.: SuGaR: Surface-aligned gaussian splatting for efficient 3d mesh reconstruction and high-quality mesh rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024). URL: https://openaccess.thecvf.com/content/CVPR2024/html/Guedon_SuGaR_Surface-Aligned_Gaussian_Splatting_for_Efficient_3D_Mesh_Reconstruction_and_CVPR_2024_paper.html. 1
- [KKLD23] KERBL B., KOPANAS G., LEIMKÜHLER T., DRETTAKIS G.: 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics* 42, 4 (2023). doi:10.1145/3592433. 1
- [SWL*24] SHAO Z., WANG Z., LI Z., WANG D., LIN X., ZHANG Y., FAN M., WANG Z.: Splattingavatar: Realistic real-time human avatars with mesh-embedded gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2024). URL: https://openaccess.thecvf.com/content/CVPR2024/papers/Shao_SplattingAvatar_Realistic_Real-Time_Human_Avatars_with_Mesh-Embedded_Gaussian_Splatting_CVPR_2024_paper.pdf. 1